# Atomics and the Memory Model

Atomic operations, compare-and-swap, memory orders and happens-before, the ordering patterns that matter, and thread-safe data structures from fine-grained locks to lock-free. Builds on Multithreading.

## Atomic operations and `compare_exchange`

`std::atomic<T>` makes operations on **one variable** indivisible, usually without blocking. The increment is now one step.

```
#include <atomic>

std::atomic<int> counter{0};
counter++;                                  // one indivisible read-modify-
write
counter.fetch_add(1);                       // same thing, explicit
int v = counter.load();                     // atomic read
counter.store(5);                           // atomic write
int old = counter.exchange(9);              // write 9, get the previous
value, all in one step

std::atomic<bool> stop{false};              // a very common use: a shutdown
flag
```

| USE AN ATOMIC FOR | DO **NOT** USE IT FOR |
| --- | --- |
| a counter, flag, swapped pointer, statistics | an invariant over **several** variables: use a mutex |

### Compare-and-swap (CAS)

`compare_exchange(expected, desired)`: "if the value still equals `expected`, set it to `desired`; otherwise load the current value into `expected` and fail". It is the building block of every lock-free update.

```
int expected = x.load();
while (!x.compare_exchange_weak(expected, expected * 2)) {
    // someone changed x: `expected` now holds the current value, so
recompute and retry
}
```

| | `COMPARE_EXCHANGE_WEAK` | `COMPARE_EXCHANGE_STRONG` |
|---|---|---|
| spurious failure | **may** fail even when the value matched | fails only if the value really differed |
| use | inside a retry **loop** (cheaper on some CPUs) | a single attempt, no loop |

Compiled: a CAS with a stale `expected` returned false and refreshed `expected` to the current value (99).

### `atomic_flag` and a spinlock

```
struct Spinlock {
    std::atomic_flag f = ATOMIC_FLAG_INIT;
    void lock()   { while (f.test_and_set(std::memory_order_acquire)) { /*
spin */ } }
    void unlock() { f.clear(std::memory_order_release); }
};
```

`atomic_flag` is the one type guaranteed lock-free. Four threads each doing 50,000 increments under this spinlock reached exactly 200,000 (compiled). A spinlock burns a core while waiting: fine for tiny critical sections on dedicated cores, bad when the lock holder can be descheduled (use a mutex).

> **VOLATILE IS NOT THREAD-SAFE** It only stops the compiler optimising an access away. No atomicity, no ordering. Use `std::atomic`.

- `is_lock_free()` says whether the type is done with real atomic instructions; big structs fall back to a hidden lock.
- **ABA problem:** a CAS can succeed when the value went A to B and back to A, although the world changed meanwhile. Fix with a version counter next to the value, or hazard pointers.

---

## Memory orders and happens-before

Compilers and CPUs **reorder** memory operations, and each core has buffers and caches, so one thread's write may become visible to another thread *later*, or *after* a write that came after it. A plain `int` gives no guarantee. Atomics let you say how much ordering you need.

### The one rule to learn: release and acquire

```
int data = 0;
std::atomic<bool> ready{false};

// producer thread
data = 42;
ready.store(true, std::memory_order_release);

// consumer thread
while (!ready.load(std::memory_order_acquire)) { }
std::cout << data;                        // guaranteed to print 42
```

- A **release store** publishes: every write the thread made *before* it becomes visible to any thread that reads the stored value with an **acquire load**.
- The release/acquire pair **synchronizes-with**, and that creates a **happens-before** edge: `data = 42` happens before the consumer's read, so the read is not a data race and sees 42.
- **happens-before** is the formal "must be visible" relation. Within one thread it is program order; across threads it comes only from synchronization (atomics, mutexes, `join`, futures).

What the consumer can print in the publish-with-a-flag example above, for each pair of memory orders:

| `ready.store(true, ...)` | `ready.load(...)` | WHAT THE CONSUMER PRINTS |
|---|---|---|
| `release` (or `seq_cst`) | `acquire` (or `seq_cst`) | always **42** |
| `relaxed` | `acquire` | may print **0**: the flag can become visible before the data |
| `release` | `relaxed` | may print **0**: nothing guarantees the data is visible after the load |
| `relaxed` | `relaxed` | may print **0** |

### The orders

| ORDER | GUARANTEE | USE |
|---|---|---|
| `relaxed` | only atomicity (and a single total order for *that* variable) | counters and statistics nobody synchronizes on |
| `acquire` (loads) | nothing after it moves before it; sees what the matching release published | reading a flag or pointer that publishes data |
| `release` (stores) | nothing before it moves after it; publishes earlier writes | setting that flag or pointer |
| `acq_rel` (read-modify-write) | both at once | `fetch_sub` on a reference count, a lock hand-over |
| `seq_cst` (default) | acquire/release **plus** one global order of all `seq_cst` operations that every thread agrees on | when in doubt: it is the only order that is easy to reason about |

> `X86 HIDES BUGS` x86 already gives release/acquire for ordinary loads and stores, so wrong orderings often "work" on a developer's PC and fail on ARM (phones, Apple silicon, many servers). The table of outcomes above shows what the **C++ model allows**, not what x86 does.

- Both sides must take part: a release store read by a **relaxed** load publishes nothing.
- A **fence** (`std::atomic_thread_fence`) gives the ordering without tying it to one variable. Know it exists; reach for acquire/release on the atomic first.
- Without any synchronization, two threads touching the same non-atomic variable (one writing) is a **data race: undefined behaviour**, not merely "stale data".

> KEY POINT "What does acquire/release guarantee?" Everything written before the release is visible after the matching acquire. Say *synchronizes-with* and *happens-before*, and that `seq_cst` is the safe default.

---

## Ordering patterns that matter

| PATTERN | ORDERS | WHY |
|---|---|---|
| **Publish with a flag or pointer** | store `release`, load `acquire` | the data written before the store is visible after the load |
| **Statistics counter** | `relaxed` `fetch_add` | only the final total matters; no other data depends on it |
| **Reference count** | increment `relaxed`, decrement `acq_rel` | adding a reference needs no ordering; the thread that drops the last one must see every other thread's writes before it deletes |
| **Spinlock** | lock `acquire`, unlock `release` | a critical section must not leak out of the lock in either direction |
| **Shutdown flag** | `relaxed` if nothing else depends on it, else `release` / `acquire` | a flag alone only needs to be seen eventually |
| **SPSC queue indices** | own index `relaxed`, other's `acquire`, publish `release` | see the ring buffer in Performance & Memory |

```
struct Counted {                                                    // like
shared_ptr's control block
    std::atomic<int> refs{1};
    void addRef()  { refs.fetch_add(1, std::memory_order_relaxed); }
    void release() {
        if (refs.fetch_sub(1, std::memory_order_acq_rel) == 1)    // the last
one out sees everyone's writes …
            delete this;                                            //  … before
destroying the object
    }
};
```

Compiled: five owners on four threads, the object was destroyed exactly once; a 2,000-round release/acquire publish test read a wrong value 0 times.

### Double-checked locking (the classic trap)

```
std::atomic<Widget*> inst{nullptr};
std::mutex m;
Widget* get() {
    Widget* p = inst.load(std::memory_order_acquire);            // 1st
check, no lock
    if (!p) {
        std::lock_guard lk(m);
        p = inst.load(std::memory_order_relaxed);                // 2nd
check, under the lock
        if (!p) { p = new Widget; inst.store(p, std::memory_order_release); }
// publish a FULLY built object
    }
    return p;
}
```

Without the release/acquire pair, another thread could see the pointer before the constructor's writes. In practice use a function-local `static` or `std::call_once` and let the compiler do this.

> **BE HONEST ABOUT WHAT YOU KNOW** Start with a mutex or `seq_cst`, measure, and weaken orders only when you can say which happens-before edge each one provides. "I use acquire/release for publication, relaxed for counters" is a sound rule of thumb.

## Thread-safe data structures

Three levels, in the order you should try them:

| LEVEL | HOW | EXAMPLE |
|---|---|---|
| **1. One coarse lock** | a mutex around the whole structure | a stack or map with a mutex (see the interface races in Multithreading) |
| **2. Fine-grained locks** | different mutexes for different parts, so threads that touch different parts do not wait | a queue with a head mutex and a tail mutex; a hash table with one mutex per bucket |
| **3. Lock-free** | atomics and CAS loops, no mutex | a Treiber stack, an SPSC ring buffer |

### Fine-grained: a queue with two locks

```
template <typename T> class TwoLockQueue {
    struct N { std::shared_ptr<T> data; std::unique_ptr<N> next; };
    std::mutex headM_, tailM_;                      // producers take tailM_,
consumers take headM_
    std::unique_ptr<N> head_;
    N* tail_;
public:
    TwoLockQueue() : head_(new N), tail_(head_.get()) {}   // a DUMMY node:
head and tail never meet on the same node
    void push(T v);                                         // locks tailM_
only
    std::optional<T> tryPop();                              // locks headM_
only (and tailM_ briefly to read the tail)
};
```

The **dummy node** is the trick: with it, a push only touches the tail and a pop only the head, so one producer and one consumer never block each other. Compiled: a producer pushing 1 to 1000 and a consumer popping them summed to 500500.

### Read-mostly: a lookup table

A price table read by many threads and updated rarely: `std::shared_mutex` with `shared_lock` for readers and `unique_lock` for writers (Other mutex types). Return **copies** or small values, never references into the table.

### Lock-free: a stack push

```
struct Node { int value; Node* next; };
std::atomic<Node*> head{nullptr};

void push(int v) {
    Node* n = new Node{v, head.load(std::memory_order_relaxed)};
    while (!head.compare_exchange_weak(n->next, n, std::memory_order_release,
std::memory_order_relaxed)) {}
    // on failure compare_exchange_weak reloads n->next with the current
head: just retry
}
```

Compiled: four threads pushing 1000 nodes each gave 4000 nodes, no lock. **Pop is the hard part**: another thread may pop and delete the node you are reading (**memory reclamation**), and the ABA problem can corrupt the list. Real implementations need hazard pointers, epochs or a counted pointer.

> LOCK-FREE IS NOT FASTER BY DEFAULT A contended CAS loop can be slower than a mutex, and a bug shows up once a month on a customer machine. Use a mutex first, measure, and use a proven library (or the SPSC ring when one producer and one consumer is enough) before writing your own.

- **Wait-free** (every thread finishes in a bounded number of steps) is stronger than **lock-free** (some thread always makes progress) which is stronger than **obstruction-free**.
- Design the **interface** first: whole operations (`tryPop` returning an `optional`), not `empty()` then `pop()`.

> FURTHER READING
>
> **Effective Modern C++ (Meyers)**
>
> - Item 40, Use std::atomic for concurrency, volatile for special memory (PDF pages 289 to 297)
>
> **C++ Concurrency in Action (Williams)**
>
> - 5.2 Atomic operations and types in C++ (atomic_flag, atomic<bool>, pointers, integers, free functions) (PDF pages 151 to 164)

- 7.1 to 7.3 Lock-free data structures: definitions, a lock-free stack, memory reclamation, the ABA problem (PDF pages 229 to 274)
- 5.1 Memory model basics (objects, memory locations, modification orders) (PDF pages 148 to 150)
- 5.3 Synchronizing operations and enforcing ordering (synchronizes-with, happens-before, the memory orders, release sequences, fences) (PDF pages 165 to 196)
- 6.2 Lock-based concurrent data structures (a thread-safe stack and queue, fine-grained locks) (PDF pages 199 to 216)